EXSOIL NSF PROTOTYPE · BENCHMARKS

CTSM container performance baseline — KONZ

Wall-clock baseline for a CLM run at one NEON tower site: build cost, model throughput, a cost model, the run-length boundary, and parallelism.

June 17, 2026 · Steven Wangen, Data Science Institute · **Draft** · Phases 0–2 measured

118 s
One-time build (compile CLM)

~6 min
Full 7-yr run (83 mo), build reused

3.2 s
Per simulated month (steady state)

~8 min
Full run, fresh container

Purpose

A wall-clock baseline for running a CLM simulation at one NEON tower site in the rebuilt container, to size researcher expectations, project campaign costs, and inform delivery decisions (local vs. hosted, prebuilt vs. runtime build). KONZ (Konza Prairie, KS) is the site already validated end-to-end in the rebuild, so it is an apples-to-apples reference.

Container access

The image is published to GitHub Container Registry with both Intel/AMD and Apple Silicon variants. Docker selects the correct architecture automatically.

```
docker pull ghcr.io/uw-madison-dsi/exsoil-nsf-prototype-refactor:v2.0.0-rc4
docker run --rm -p 8888:8888 ghcr.io/uw-madison-dsi/exsoil-nsf-prototype-refactor:v2.0.0-rc4
```

Registry	`ghcr.io/uw-madison-dsi/exsoil-nsf-prototype-refactor`
Current tag	`v2.0.0-rc4`
Architectures	`linux/amd64` (Intel/AMD), `linux/arm64` (Apple Silicon)
Image size	~14.7 GB uncompressed (~7 GB compressed pull)
Includes	CTSM 5.4.043, CLM 6.0, CIME 6.1, ~7.9 GB pre-loaded global input data, Python 3.13 + JupyterLab
Source	github.com/UW-Madison-DSI/exsoil-nsf-prototype-refactor

Global input data is embedded in the image. Only site-specific NEON tower forcing (~150 KB/month, from Google Cloud) downloads at runtime. No NCAR account or credentials needed.

Test environment

Image	`ghcr.io/uw-madison-dsi/exsoil-nsf-prototype-refactor:v2.0.0-rc4` (arm64, ~14.7 GB)
CTSM	5.4.043 (CLM 6.0, CIME 6.1)
Host	Apple M3 Pro, 11 cores, 36 GB RAM, macOS 26.2; Docker 29.4.0 (VM: 11 CPUs / ~7.75 GB)
Run	KONZ transient (`IHist1PtClm60Bgc`), single core (`mpiexec -n 1`), from published `finidat`

Timings are wall-clock from CIME `CaseStatus` and the `run_tower` wrapper. Input data is embedded, so there is no GDEX/network dependency.

> **Note on the image.** These timings were measured on the equivalent local build (`exsoil-full:latest`), not on the published `v2.0.0-rc4` tag directly. The two share the same CTSM 5.4.043 / CLM 6.0 / CIME 6.1 stack and embedded input data, so the numbers should carry over, but `v2.0.0-rc4` has not been separately re-timed.

Results

MEASUREMENT	WALL-CLOCK
Base-case build (compile CLM) — **one-time**	**118 s**
Model run, 12 simulated months	108 s
Model run, full 83 months (2018-01 → 2024-11)	**337 s**
Full run, `run_tower` wrapper (clone → stage → run → archive)	**362 s (~6 min)**

The full run reused the base case via `--base-case`, so **no recompile occurred**. `case.run` reported success; coverage is 83 monthly history files with a clean restart at 2024-12-01.

Cost model

The two run lengths (12 mo → 108 s, 83 mo → 337 s) fit:

`model run ≈ 69 s fixed init + 3.2 s × (simulated months)`

Steady state ≈ 39 s/simulated-year. A naive linear projection from a short run overestimates — the short run carries the full ~69 s init cost.

Full campaign cost: ~6 min with the base case already built (hosted / repeat use); ~8 min from a fresh container (adds the one-time 118 s build).

STOP_N boundary behavior (gotcha)

KONZ has 84 forcing months (2018-01 → 2024-12), but a transient run can only cleanly integrate **83** (`STOP_N=83`). `STOP_N=84` integrates the full 7 years and then aborts on the *final* timestep:

```
(shr_strdata_advance) Stream 1: LB ymd = 20241231  UB ymd = 20250101
(shr_stream_findBounds) ERROR: rDateIn ≥ rDategvd limit true
```

The run ends at 2025-01-01 00:00 and DATM needs a forcing record beyond that to bracket the last interpolation — a temporal off-by-one, not a data gap or performance issue. **Guidance:** stop at least one forcing timestep before the last record. Also note `run_tower`'s exit code is unreliable (returns 0 on *submit*); the real status is `case.run success` / `error` in `CaseStatus`.

The build step: why it runs at runtime, and how often

CLM is compiled at container *runtime* rather than baked into the image, by deliberate choice (ADR-0007): the arm64 image is built under QEMU emulation in CI, which handles I/O well but compiles large codebases poorly. The image ships source + toolchain, and CLM compiles natively on first use (~118 s). The cost amortizes to ~zero because the executable persists and is reused.

CLM compiles **once** into a base case; every site/run is a `create_clone(keepexe=True)` that reuses the executable.

- **Recompiles only when:** no base case exists yet (fresh container); the compset changes (transient `IHist1PtClm60Bgc` vs. spin-up `I1PtClm60Bgc` are separate builds); build config changes (compiler, resolution, FATES/BGC, `SourceMods`); or `--overwrite` / `--clean`.
- **No recompile for:** different site, run length/dates (`STOP_N`, `DATM_YR_*`), `finidat`, history output, or forcing/parameter perturbations (all runtime config).
- **Persistence:** the executable lives in the base case `bld/`. On a mounted volume or hosted instance it survives and is reused via `--base-case`; in an ephemeral container layer it is lost and the next run recompiles. This is the main lever for eliminating the build cost.

Parallelism

- **Within one run — no.** A NEON run is a single grid column, so there is nothing to decompose (hence `mpi-serial`, `mpiexec -n 1`), and the sequential timestepping can't be parallelized. The ~3.2 s/month rate is a serial floor; extra cores idle.
- **Across runs — yes, embarrassingly parallel.** Independent sites and calibration/perturbation ensembles all share the one `cesm.exe`, so the pattern is "clone N cases and run concurrently." Throughput ≈ single-run cost ÷ concurrent runs.
- **Constraints:** `run_tower` runs sites serially in `--no-batch` mode (concurrency needs separate processes/containers or CIME batch submit); on one machine, memory (the ~7.75 GB VM), not cores, is the likely limiter. At ensemble/multi-site scale this is a scheduler/cloud workload — the hosted-vs-distributable delivery question.

Caveats & next steps

Single host, single sample per length; the cost model is a two-point fit; throughput is from `CaseStatus` wall-clock (CIME's internal timer file was not generated for these single-point runs); the amd64 path is unmeasured. Benign log noise: a global default `finidat` 404 (the NEON KONZ file is used instead) and "Model datm missing file" lines (the pre-run check listing files to download).

Optional follow-ups: spot-check a second site; measure amd64; and a concurrency micro-benchmark (2 / 4 / 8 simultaneous runs → per-run slowdown, peak memory, how many fit in the VM) to get real per-machine ensemble throughput for calibration.